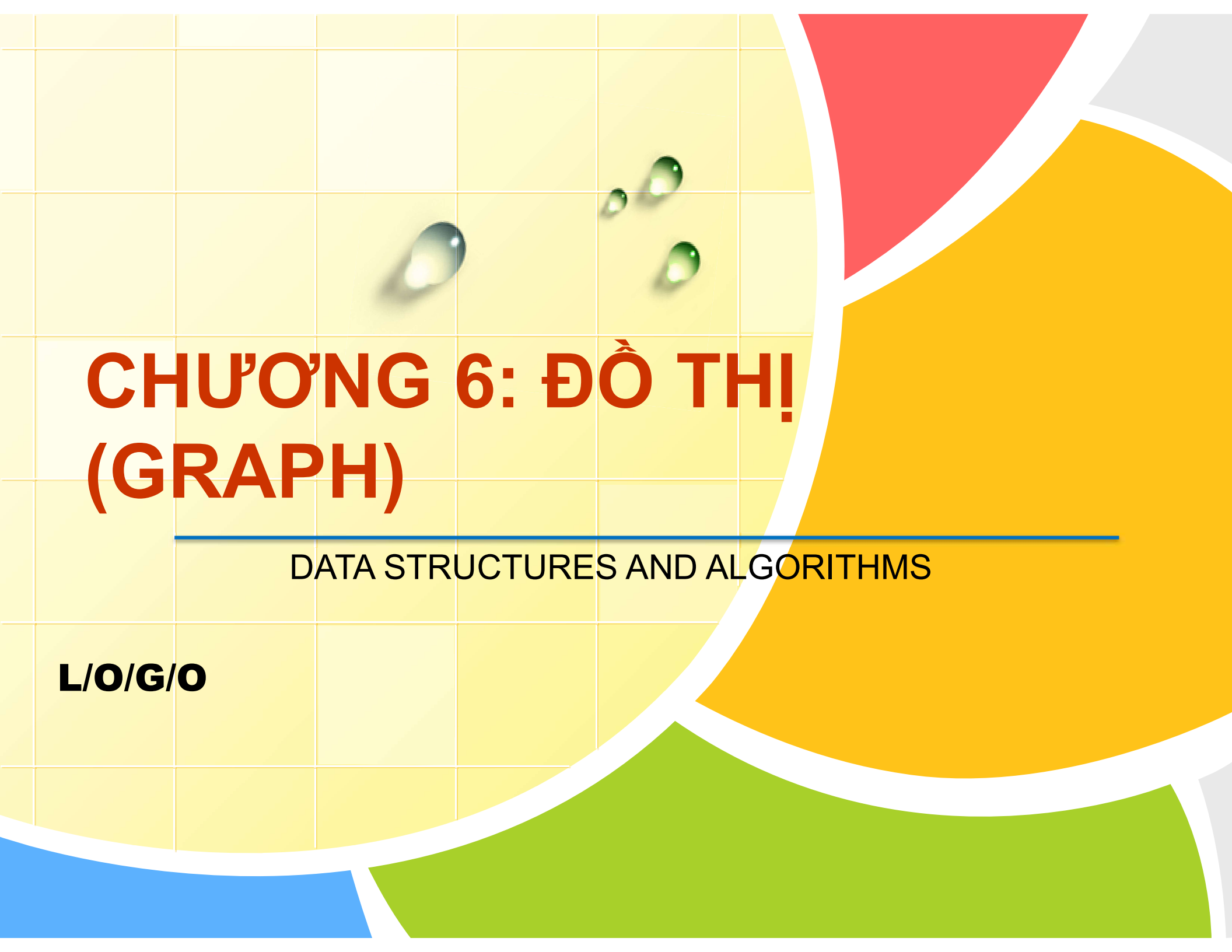




CHƯƠNG 6: ĐỒ THỊ (GRAPH)

DATA STRUCTURES AND ALGORITHMS

L/O/G/O

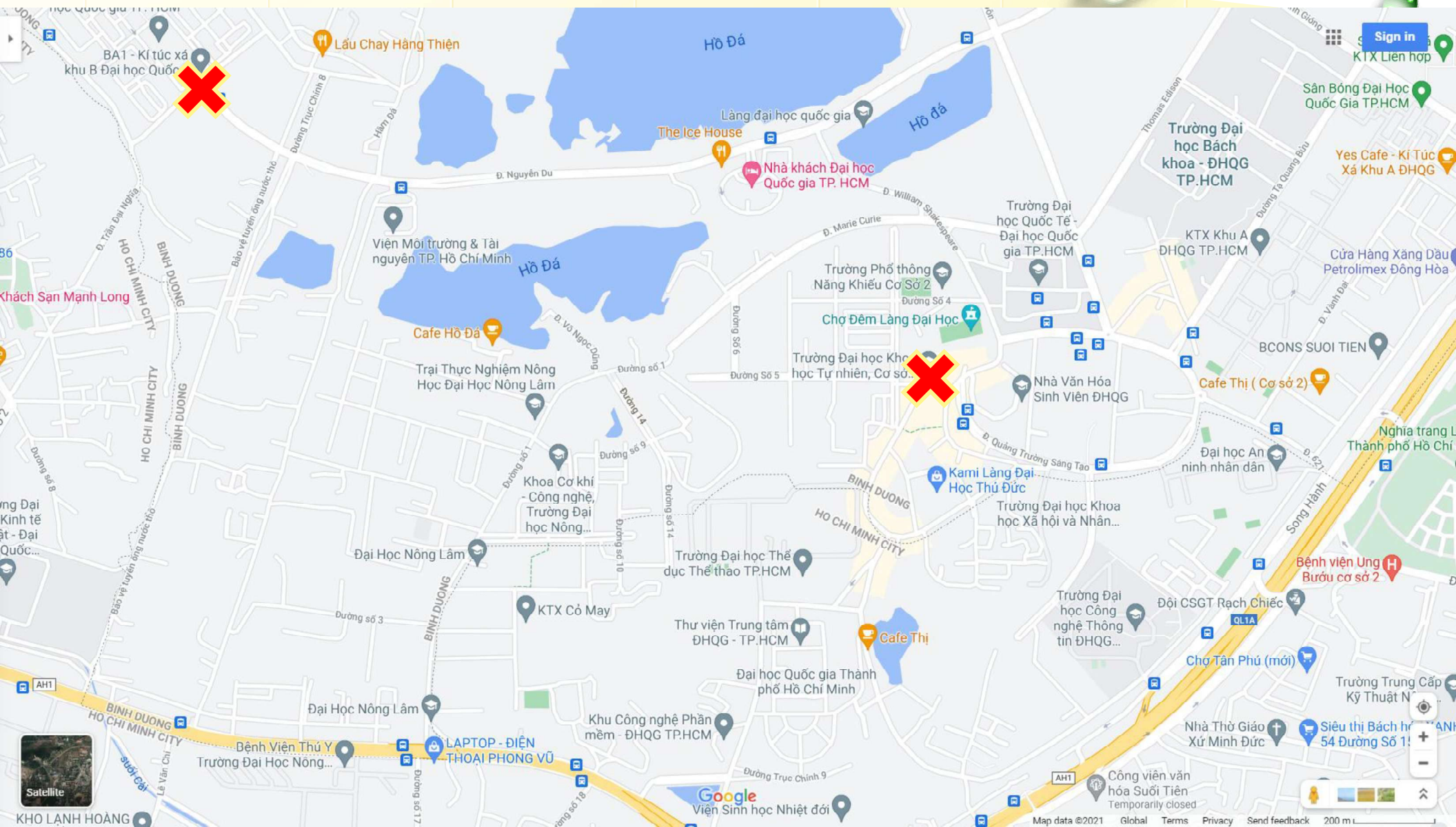
Nội dung buổi 2

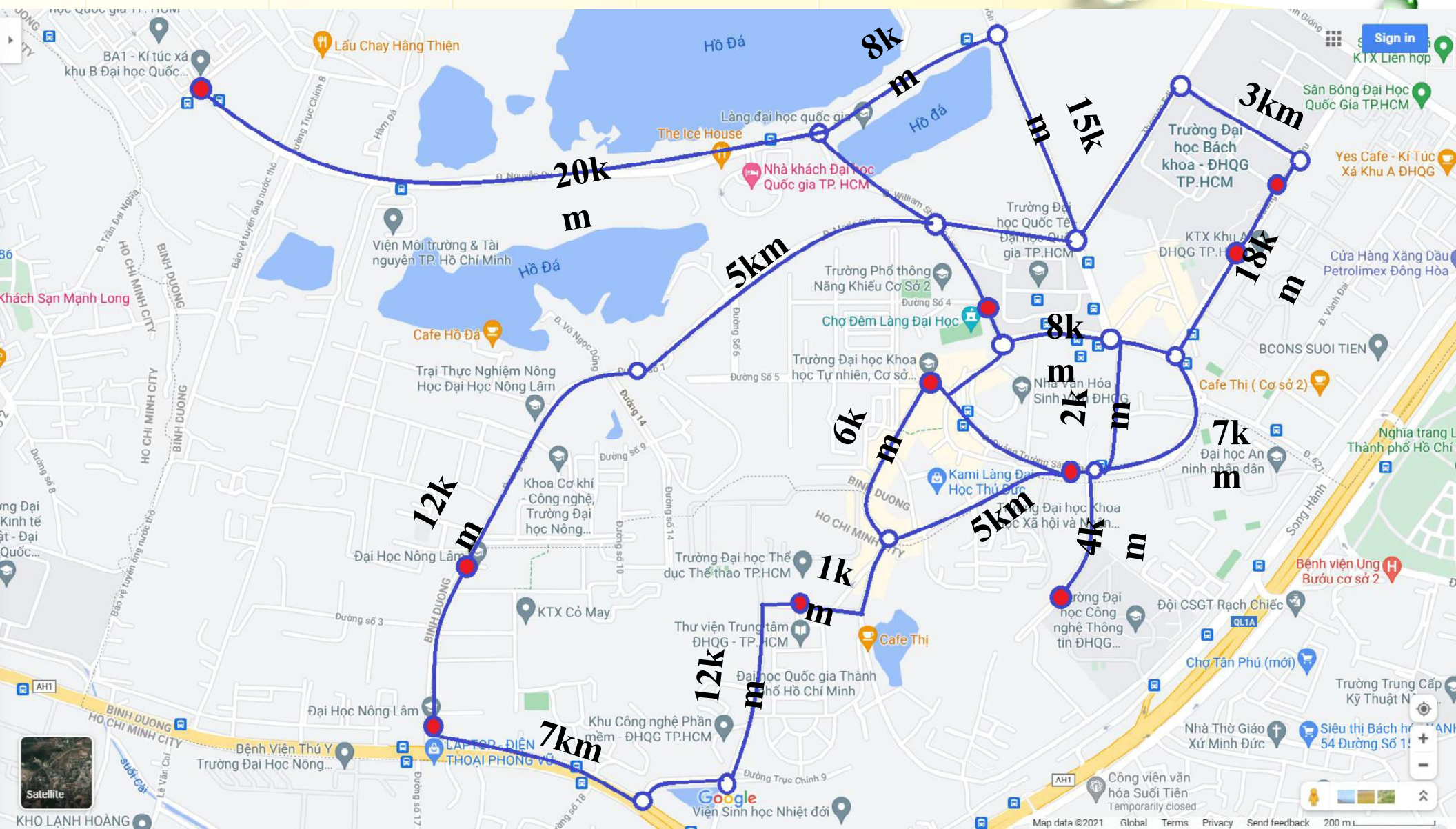
6.1 Các khái niệm trên đồ thị

6.2 Biểu diễn đồ thị trên máy tính

6.3 Duyệt đồ thị

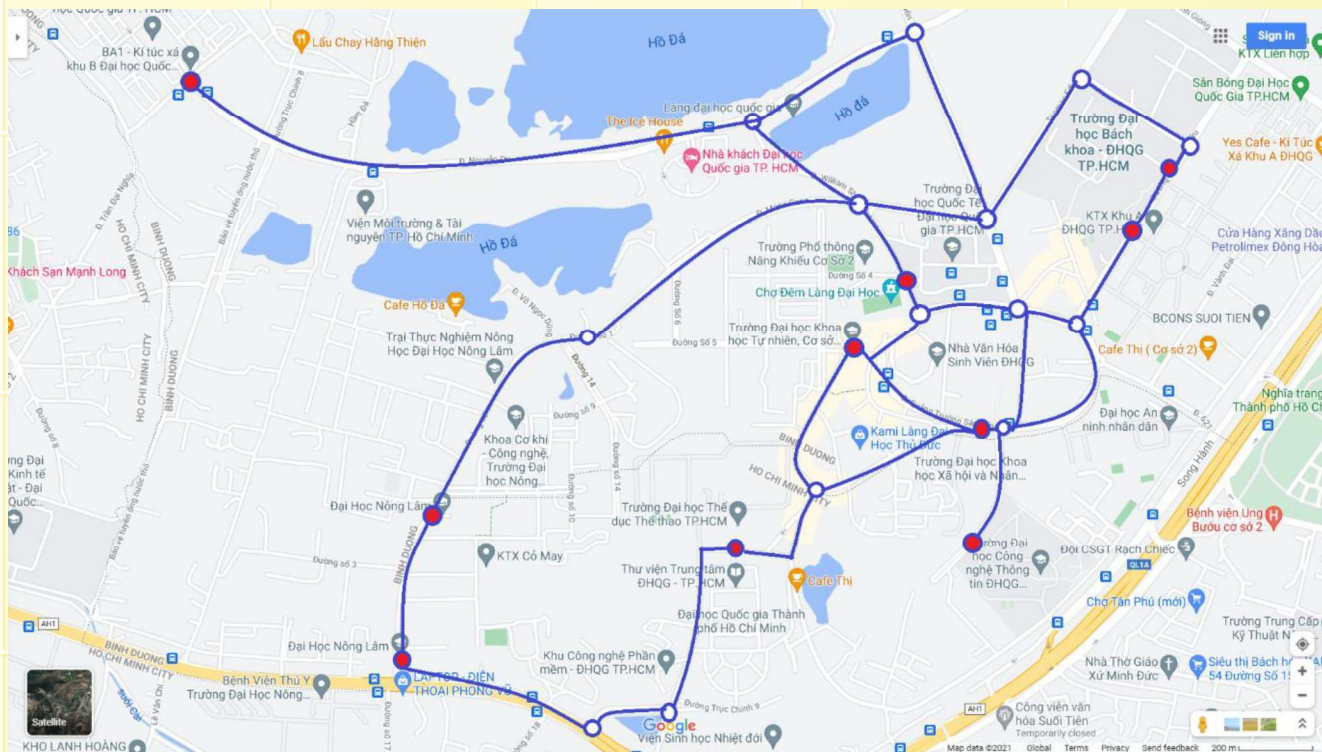
6.4 Thuật toán tìm đường đi ngắn nhất





Các bài toán

1. Tìm đường đi ngắn nhất từ một đỉnh đến một đỉnh khác.
2. Tìm đường đi ngắn nhất từ một đỉnh đến các đỉnh còn lại
3. Tìm đường đi ngắn nhất giữa các cặp đỉnh



Thuật toán Dijkstra

❖ Giới thiệu:

- Thuật toán tìm đường đi ngắn nhất từ một đỉnh xuất phát đến tất cả các đỉnh còn lại trong đồ thị có hướng, có trọng số không âm
- Được công bố lần đầu vào năm 1959
- Được đặt tên theo tên của nhà toán học và nhà vật lý người Hà Lan Edsger W. Dijkstra

Edsger Wybe Dijkstra

(/'daɪkstrə/ *DYKE-strə*)

(11/5/1930 – 6/8/2002)



- Nhà toán học, nhà vật lý, nhà khoa học máy tính người Hà Lan
- Làm việc tại Trung tâm toán học, Viện nghiên cứu quốc gia về toán học và khoa học máy tính tại Amsterdam
- Giữ chức vị giáo sư tại Đại học Kỹ thuật Eindhoven, Hà Lan.
- Nghỉ hưu năm 2000

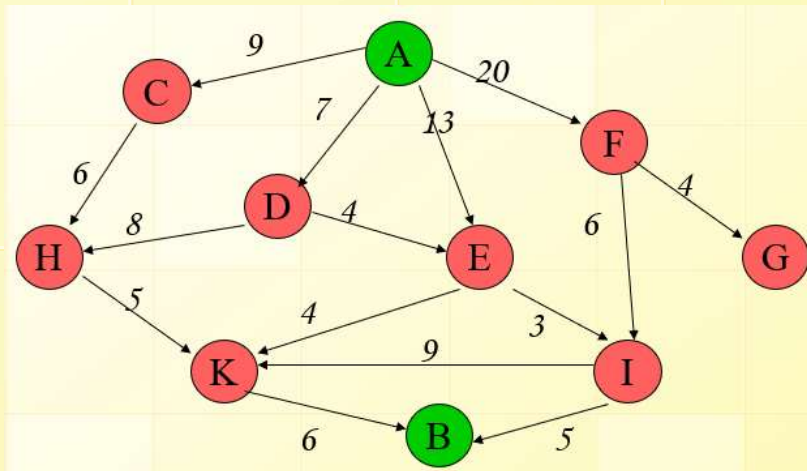
- Có các đóng góp mang tính chất nền tảng trong lĩnh vực ngôn ngữ lập trình
- Thuật toán Dijkstra, hệ điều hành THE và cấu trúc semaphore...

“Computer Science is no more about computers than astronomy is about telescopes”

Thuật toán Dijkstra

❖ Bài toán 1:

- **Input:** Đồ thị có hướng $G=(V,E,w)$, $w(i,j) \geq 0 \ \forall \ (i,j) \in E$, đỉnh nguồn s , đỉnh đích g
- **Output:** đường đi ngắn nhất từ đỉnh s đến đỉnh g và độ dài của đường đi



Thuật toán Dijkstra

❖ Ý tưởng chính:

- Hàm $d(u)$ dùng để lưu trữ độ dài đường đi (khoảng cách) ngắn nhất từ đỉnh nguồn s đến đỉnh u

$$d(u) = \min\{d(v) + w(v, u), v \in X(u)\}$$

$X(u)$ là tập tất cả các đỉnh có cạnh đi tới đỉnh u

- Đặt khoảng cách từ đỉnh nguồn s đến chính nó là 0 và đến tất cả các đỉnh khác là vô cùng
- Tiến hành lặp cho đến khi tất cả các đỉnh đã được xác định khoảng cách ngắn nhất từ s hoặc không còn đỉnh nào có thể đạt tới từ s
- Mỗi lần lặp, chọn đỉnh p chưa đi qua có giá trị $d(p)$ **nhỏ nhất**, cập nhật khoảng cách của các đỉnh kề thông qua đỉnh được chọn p

Ký hiệu

Gọi **Open** : tập các đỉnh có thể xem xét lựa chọn cho bước đi tiếp theo, các đỉnh đã được “sinh ra” nhưng chưa được chọn/duyệt, đỉnh chờ duyệt

Close : tập các đỉnh đã duyệt/đã đi qua (mỗi đỉnh chỉ được đi qua một lần duy nhất)

s : đỉnh xuất phát

g : đỉnh đích

p : đỉnh đang xét, đỉnh hiện hành

$\Gamma(p)$: tập các đỉnh kề của p

$q \in \Gamma(p)$: một đỉnh kề của p

Trong một số trường hợp, có thể sử dụng thêm **Visited**: đánh dấu các đỉnh đã được “sinh ra”, đã được bỏ vào Open

Dijkstra

❖ Giải thuật tham khảo:

Hàm $d(u)$ lưu trữ độ dài đường đi ngắn nhất từ đỉnh nguồn s đến đỉnh u

Bước 1: Khởi tạo

Open := { s };

Close := {};

$d(s) := 0$;

Bước 2: While (Open $\neq$ {})

2.1 Chọn p thuộc Open có **$d(p)$ nhỏ nhất** (xóa p ra khỏi Open). Nếu có nhiều đỉnh cùng giá trị $d(p)$ nhỏ nhất thì chọn đỉnh nào? (tự quy ước)

2.2 Nếu p là đỉnh đích thì xuất đường đi, kết thúc thuật toán

2.3 Nếu p đã duyệt rồi thì bỏ qua, không xem xét lại, trở lại đầu vòng lặp

2.4 Đánh dấu p đã duyệt qua rồi (tức thêm p vào Close)

2.5 **Mở các đỉnh kế tiếp q sau p (kề với p)**

Tìm kiếm Dijkstra

Bước 2: While (Open $\neq \{\}$)

2.5 Mở các đỉnh kế tiếp q sau p:

Với mỗi đỉnh q kề với p, nếu q không thuộc Close:

2.5.1 Nếu q đã có trong Open

và $d(q) > d(p) + \text{cost}(p, q)$ thì cập nhật đường đi mới

$$d(q) = d(p) + \text{cost}(p, q);$$

$$\text{parent}(q) = p;$$

2.5.2 Nếu q chưa có trong Open

$$d(q) = d(p) + \text{cost}(p, q);$$

$$\text{parent}(q) = p;$$

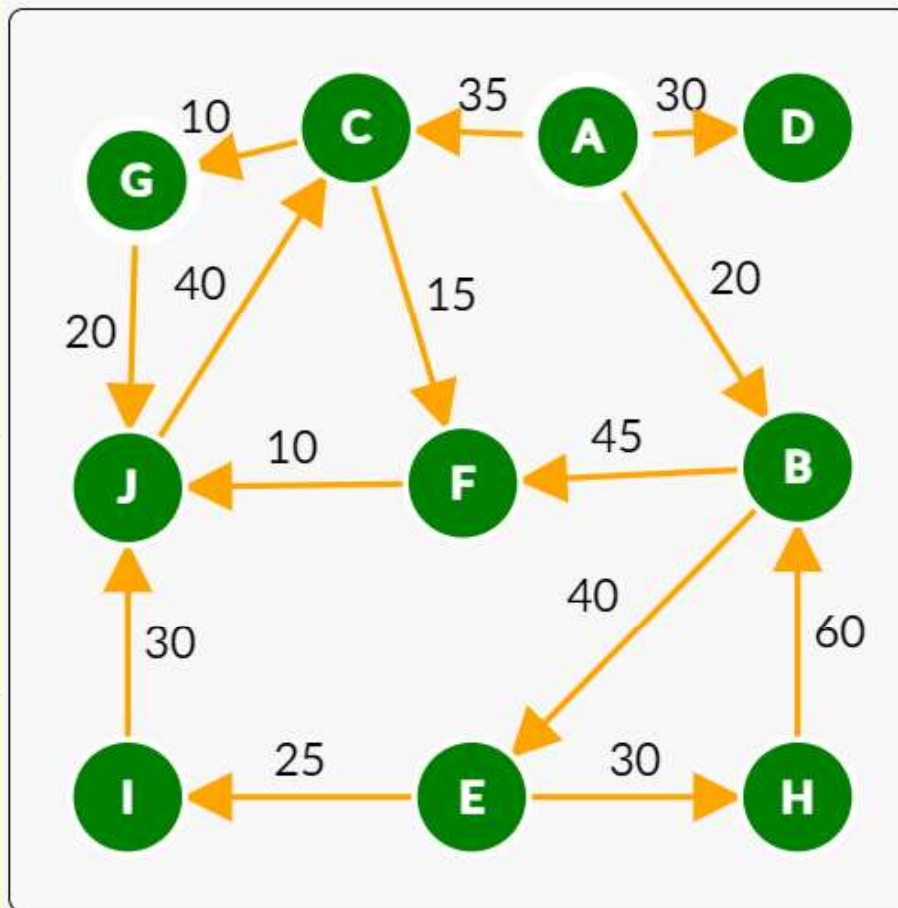
Thêm q vào Open

Câu hỏi thảo luận:

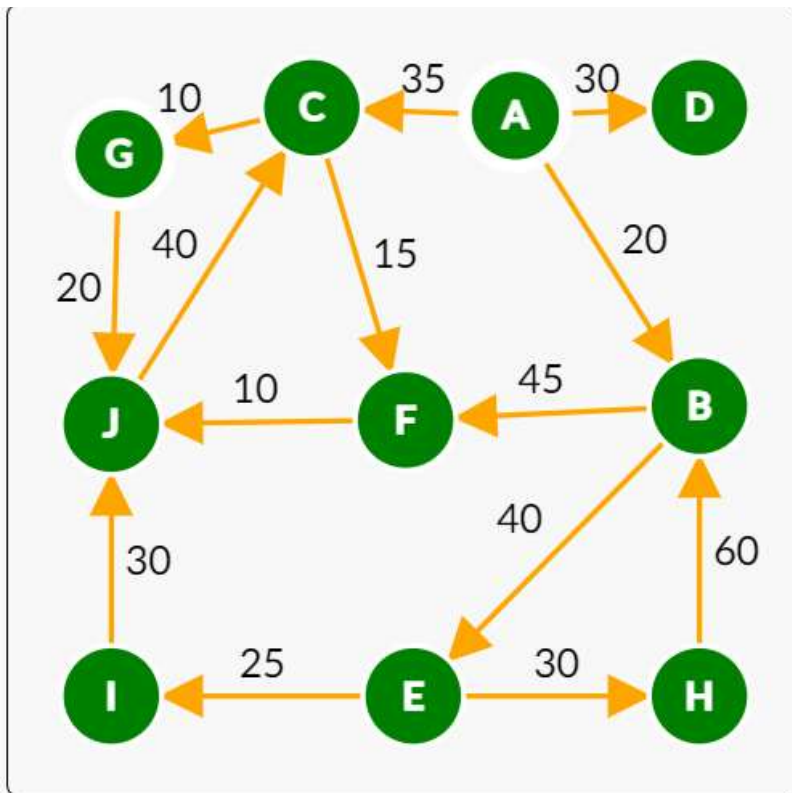
- Cách truy xuất đường đi?
- Độ phức tạp?

Bước 3: Kết luận “Không tìm thấy đường đi”

Ví dụ: Tìm đường đi ngắn nhất từ A đến J



10	14
A	B C D E F G H I J
A	B 20
A	C 35
A	D 30
B	E 40
B	F 45
C	F 15
C	G 10
E	H 30
E	I 25
F	J 10
G	J 20
H	B 60
I	J 30
J	C 40

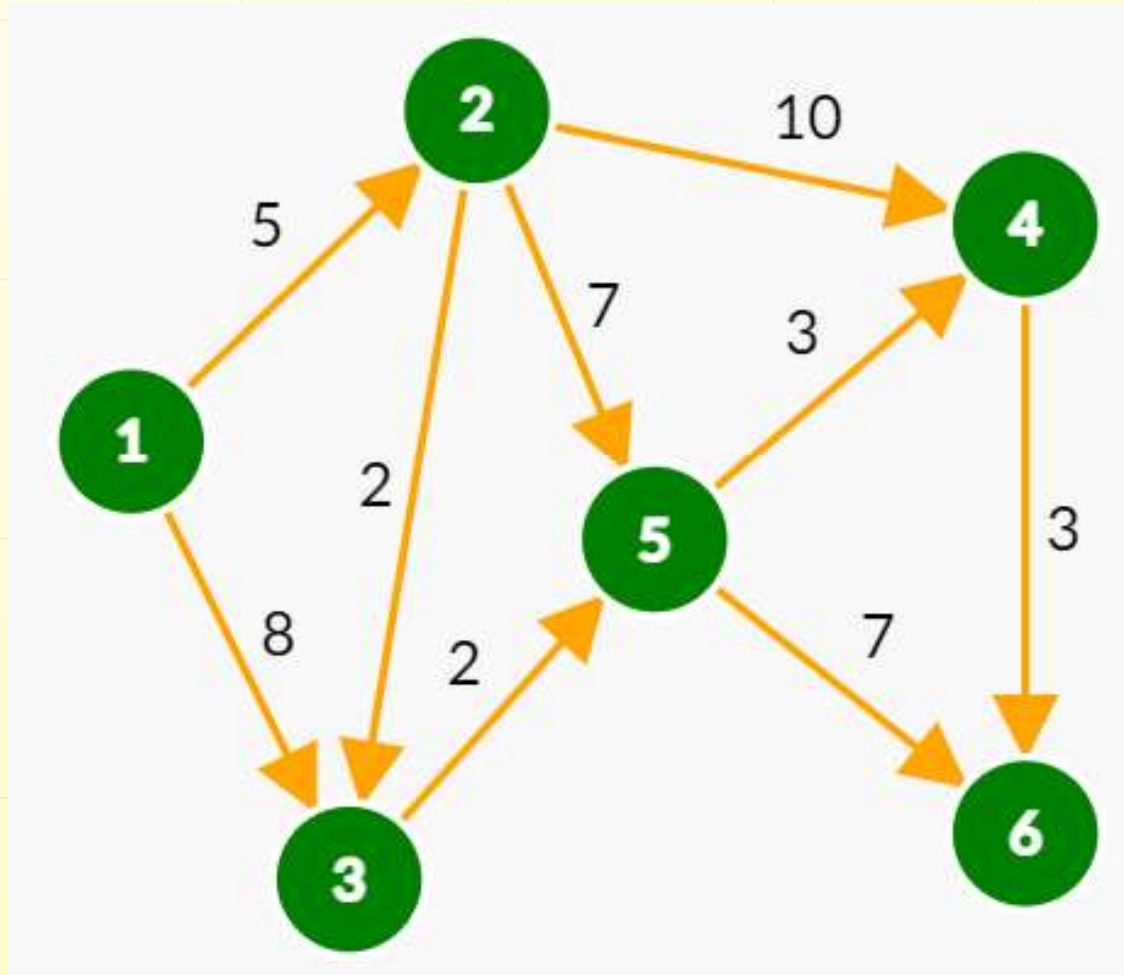


Ví dụ: Tìm đường đi ngắn nhất từ A đến J dùng Dijkstra

p	$\Gamma(p)$	Open	Close
		{(A,0)}	{}
A	{B,C,D}	{(B,20), (C,35), (D,30)}	{A}
B	{E,F}	{(C,35), (D,30), (E,60), (F,65)}	{A,B}
D	{}	{(C,35), (E,60), (F,65)}	{A,B,D}
C	{F,G}	{(E,60), (F,65) , (F,50), (G,45)}	{A,B,D,C}
G	{J}	{(E,60), (F,50), (J,65)}	{A,B,D,C,G}
F	{J}	{(E,60), (J,65) , (J,60)}	{A,B,D,C,G,F}
J			

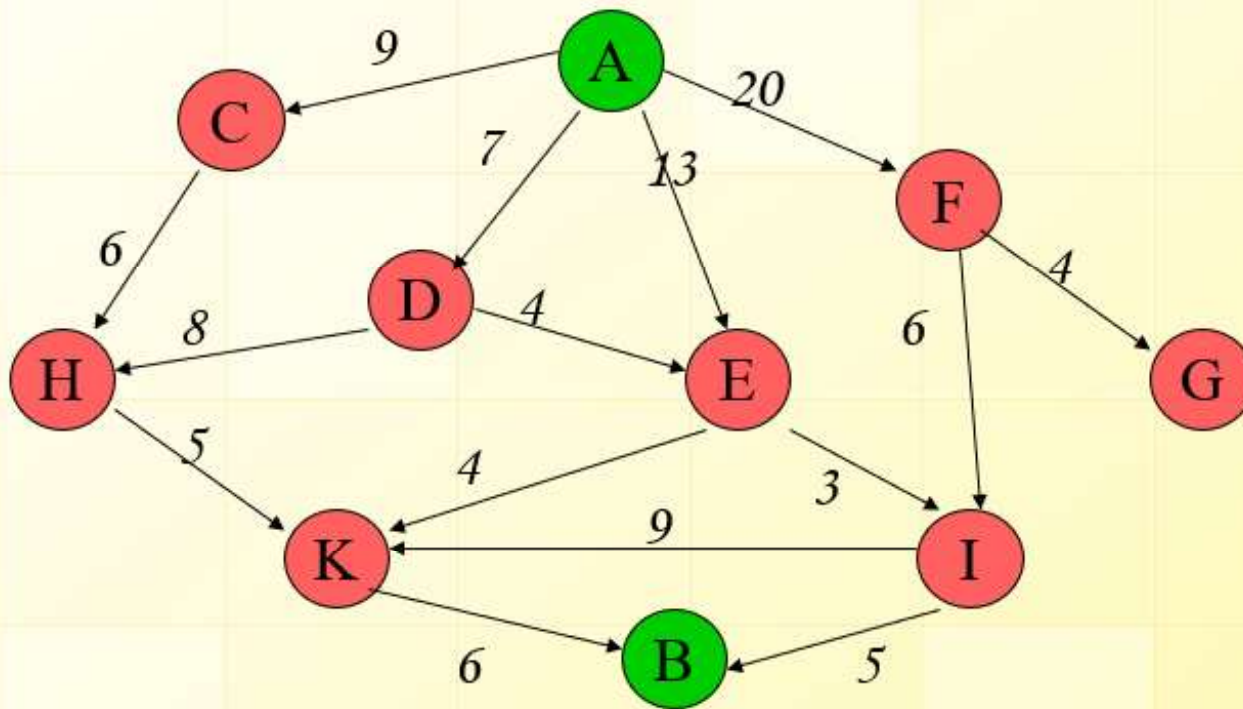
Bài tập tại lớp:

Tìm đường đi ngắn nhất từ 1 đến 6

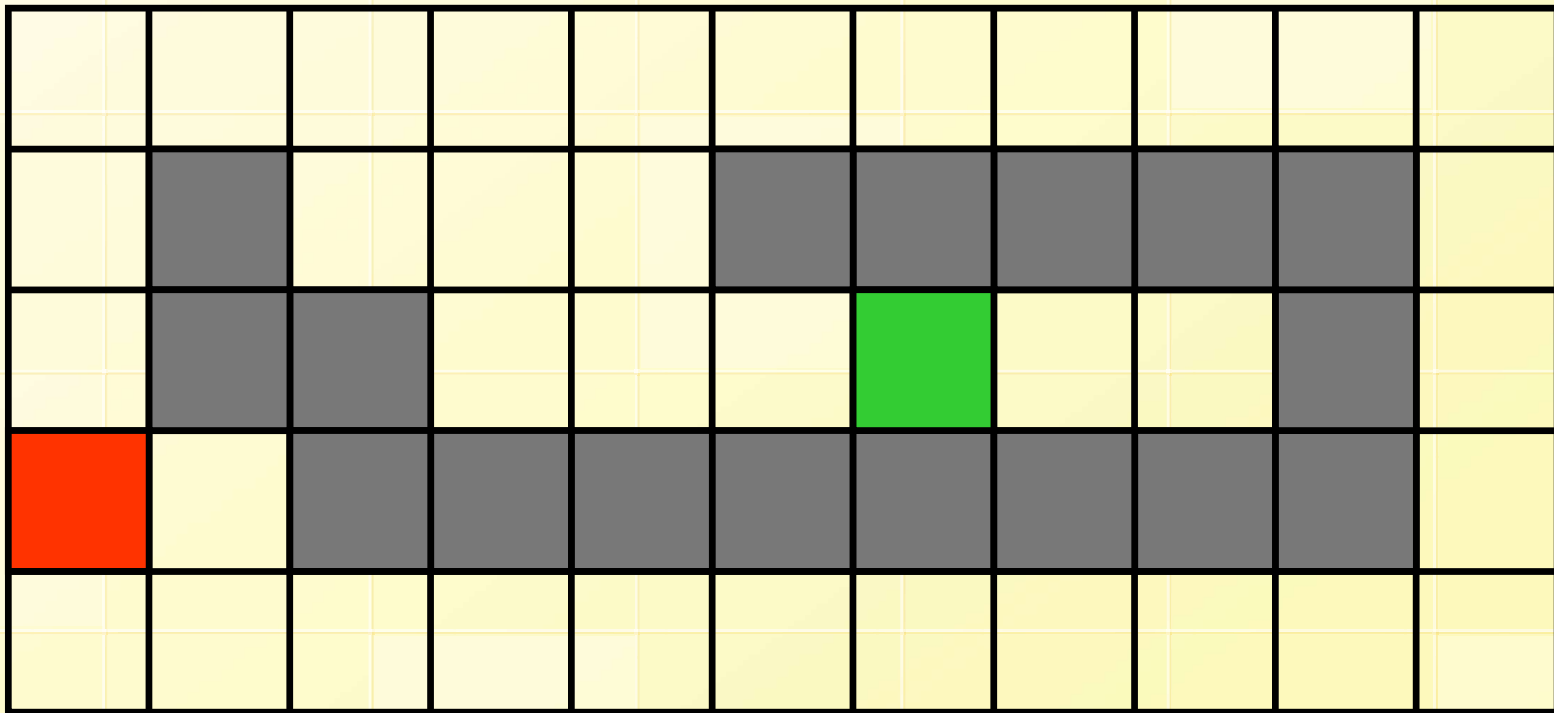


Bài tập tại lớp:

Tìm đường đi ngắn nhất từ A đến B



Bài tập tại lớp: Robot tìm đường



Tìm kiếm Dijkstra

❖ Một số gợi ý về cài đặt:

1. Tổ chức cấu trúc dữ liệu

- `vector<vector<int> > matrix;` // ma trận trọng số của đồ thị
- `vector<string> names;` // lưu danh sách các tên đỉnh (chuỗi)
- `map<string, int> toIndex;` // ánh xạ từ tên đỉnh sang index để có thể truy xuất thông tin trong ma trận kề
- `priority_queue<....>open;` // lưu các đỉnh chờ duyệt
- `vector<bool> close(v, 0);` // lưu thông tin đỉnh nào đã duyệt qua rồi
- `map<string, string> parent;` // lưu thông tin cha con, `parent[u]=v` nghĩa là cha của u và v
- `vector<int> d(v, INF);` // lưu khoảng cách (ngắn nhất) từ s đến các đỉnh khác

Priority Queue

Khởi tạo min heap từ priority queue

```
priority_queue< pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>>> open;
```

Các hàm trên Priority Queue:

- size(): Trả về kích thước của hàng đợi
- empty(): kiểm tra có rỗng hay không
- top(): Lấy phần tử có độ ưu tiên cao nhất
- pop(): Xóa phần tử có độ ưu tiên cao nhất
- push(): Thêm 1 phần tử vào

Priority queue cũng giống như queue nhưng được thiết kế đặc biệt để phần tử ở đầu luôn luôn là phần tử có độ ưu tiên lớn nhất so với các phần tử khác

Tìm kiếm Dijkstra

❖ Một số gợi ý về cài đặt:

Bước 1: `open.push({0,s}); d[s] = 0;`

Bước 2: `while (!open.empty())`

`pair<int, int> top = open.top(); open.pop();`

`int p=top.second, kc=top.first;`

`if (p == g) {found = true; break; }`

`if(close[p]==1) continue;`

`close[p] = 1;`

`for (int i = 0; i < v; i++)`

`if (matrix[p][i]>0 && close[i]== 0)`

`if (d[i]>d[p]+ matrix[p][i] )`

`d[i]=d[p]+ matrix[p][i] ; open.push({d[i],i}); parent[i] = p;`

Bước 3: Xử lý xuất đường đi hoặc kết luận không tìm thấy